

Codebase Audit & Architecture Review

Subject: Evaluation of Technical Queries & Architectural Feedback for Personal Payment Tracker Bot
Target Repository: github.com/kanagarlapranav/payment | **Evaluation Date:** September 2026

Executive Verdict: Is an Upgrade Required?

Yes, an upgrade is strongly recommended, primarily due to an active **credential exposure** and **authorization vulnerabilities**. Your developer friend's review was exceptionally sharp and grounded in genuine production risks. While the bot currently functions well for basic personal transactions, the real Telegram Bot Token was committed directly into git history, button-based callbacks lack authorization checks, and specific balance-altering commands bypass the cloud backup engine.

This report provides an itemized technical evaluation of each query, code-verified findings, and an actionable upgrade roadmap.

1. Summary Review Matrix

Query / Item	Friend's Claim	Code Verification	Severity	Action Required?	Effort
1. Leaked Bot Token check_messages.py setup_env.py	Hardcoded in plain source & git history	Confirmed Committed in affae641	CRITICAL	Immediate (Revoke & Rotate)	5 mins
2. Access Control Callbacks & Groups	Check if commands/actions restricted	Gap Found Callbacks unverified	HIGH	Yes (Add auth to callbacks)	10 mins
3. Deployment Conflict Render vs Vercel/Netlify	Incompatible architecture files present	Confirmed Polling vs Serverless	MEDIUM	Yes (Purge dead configs)	5 mins
4. Tesseract OCR Linux OS dependency	Fails on Render without Dockerfile	Shielded RapidOCR is primary	LOW	Optional (Clean setup exe)	5 mins
5. DB Persistence Ephemeral disk on Render	Risk of data loss on container restarts	Gap Found /setbalance unbacked	MEDIUM	Yes (Trigger backup on write)	5 mins
6. Keep-Alive Ping Internal vs External	Self-ping cannot revive frozen process	Confirmed Burns 744h free quota	LOW	Recommended (External cron)	10 mins
7. Utility Scripts Duplicated setup code	Redundant near-identical files	Confirmed Duplicate polling logic	LOW	Optional (Consolidate)	5 mins

2. Technical Verification & Solutions

Point 1: Hardcoded Token in Source Files & Git History (Critical)

Observation: `check_messages.py` (line 6) and `setup_env.py` (line 6) hardcoded the live production Telegram Bot Token: 8863268724:AAFcDfdgTXas2E6OnNIQj9mRIwRzQ8WV94.

Git Verification: Confirmed. Both files were included in initial commit `affae64155db5ab9eddcce8afe31bf6379790193` and pushed to remote origin/main. Because Git retains full commit history, removing the lines in a future commit does not protect the token — any user or crawler viewing public commit history can extract it.

Impact: Full takeover of bot identity, access to payment screenshots, and potential tampering with financial data.

Solution:

1. Revoke the token immediately via Telegram **@BotFather** (/mybots → select bot → API Token → Revoke).
2. Place the new token into Render environment variables and local `.env`.
3. Refactor `check_messages.py` and `setup_env.py` to load from `os.getenv('TELEGRAM_BOT_TOKEN')`.

Point 2: Command & Callback Authorization Gaps (High)

Observation: Your friend advised checking whether command access is gated to authorized users.

Code Verification:

- **Commands are protected:** Text commands (`/balance`, `/history`, `/delete`, etc.) correctly call `is_authorized(update)`.
- **Vulnerability in Callbacks:** In `bot/handlers.py:171`, `handle_callback_query()` **does not check authorization**. Any user who can see or tap inline buttons can execute `confirm_tx`, `cancel_tx`, or `delete_confirm`.
- **Group ID Broadness:** In `bot/commands.py:23`, `is_authorized` returns `True` if `chat_id == TELEGRAM_GROUP_ID`. If the bot is in a group with multiple users, any member can execute commands.

Solution: Insert `if not await is_authorized(update): return` at line 174 of `bot/handlers.py`, and restrict destructive commands (`/setbalance`, `/delete`) strictly to `TELEGRAM_USER_ID`.

Point 3: Conflicting Deployment Architectures (Render vs Vercel / Netlify)

Observation: The repository contains conflicting deployment files: `render.yaml` + Procfile (daemon long-polling) vs `vercel.json` + `netlify.toml` + `api/index.py` (serverless webhook).

Code Verification: Confirmed. Long-polling and serverless are architecturally incompatible for this application. In serverless functions, SQLite files do not persist across invocations, background threads are terminated on response return, and free execution timeouts (10s) trigger OCR failures.

Danger: Running `set_webhook.py` permanently disables Render's long-polling until explicitly deleted from Telegram.

Solution: Standardize on Render. Remove `vercel.json`, `netlify.toml`, `set_webhook.py`, and `api/index.py`.

Point 4: Tesseract OCR vs RapidOCR on Render Cloud

Observation: Friend noted that `pytesseract` requires OS-level binary `apt-get install tesseract-ocr`, which Render's native Python runtime lacks.

Code Verification: Your code in `ocr/engine.py:32-43` already uses **RapidOCR (ONNX Runtime)** as the primary OCR engine. RapidOCR runs purely within Python without external OS dependencies. Tesseract is only a fallback and is skipped safely via `shutil.which('tesseract')`.

Cleanup: `tesseract_setup.exe` (50 MB installer) is sitting in the root directory, and `pytesseract` is in `requirements.txt`. Both can be cleaned up to streamline repo size and deployment times.

Point 5: SQLite Database Persistence & Backup Reliability

Observation: Render free-tier disk is ephemeral and resets on every redeploy/restart. Backups must run on every write.

Code Verification: You implemented a resilient backup mechanism via `services/backup_service.py` that backs up to Telegram and restores on startup.

Identified Vulnerability: Backups trigger after image transactions, button confirms, and deletes, but **NOT** after `/setbalance <amount>` in `bot/commands.py:530`. If Render restarts after a balance adjustment, the new balance is reverted to the prior backup.

Solution: Add `asyncio.create_task(backup_to_telegram(context.bot))` inside `setbalance_command()`.

Point 6 & 7: Keep-Alive Pinging & Script Deduplication

Keep-Alive & Free Quota: `app.py:91-103` runs an internal daemon pinging `RENDER_EXTERNAL_URL` every 10 minutes. Render Free Web Services allocate **750 free instance hours/month**. Running `24/7/365` consumes 744 hours (99.2% of account allowance). If the Python process crashes, the internal thread dies with it. An external uptime monitor (e.g. *cron-job.org* or *UptimeRobot*) is more resilient.

Script Deduplication: `check_messages.py` (times out after 15 attempts) and `setup_env.py` (infinite loop) duplicate 90% of the same code. They can be merged into a single clean `get_chat_ids.py` utility.

3. Recommended Upgrade Action Plan

Phase	Concrete Action Items	Timing & Priority	Risk If Postponed
Phase 1 Security Hardening	1. Revoke leaked Telegram token via @BotFather. 2. Update token in local .env & Render dashboard. 3. Remove token string from <code>check_messages.py</code> & <code>setup_env.py</code> . 4. Add <code>is_authorized()</code> to <code>handle_callback_query()</code> in <code>bot/handlers.py</code> .	Immediate (Within 24-48 hrs)	Bot takeover & data leakage
Phase 2 Data Integrity	1. Add Telegram backup trigger to <code>/setbalance</code> . 2. Remove dead serverless configs (<code>vercel.json</code> , <code>netlify.toml</code> , <code>api/</code>). 3. Remove 50MB <code>tesseract_setup.exe</code> from repository.	Next Update (When convenient)	Loss of balance state on Render restart
Phase 3 Maintenance	1. Connect free external uptime ping (<i>cron-job.org</i> / <i>UptimeRobot</i>). 2. Consolidate setup scripts into a single <code>get_chat_ids.py</code> .	Optional (Maintenance cycle)	Exhaustion of free monthly Render hours

Summary & Next Steps: Since you confirmed the bot is currently working well for your needs, you can keep running it without immediate disruption. However, because the bot token is committed to Git history, prioritizing **Phase 1 (Token Rotation & Callback Auth)** will ensure your private financial tracker remains completely secure.